DAILY FUSION CODE · WEEK 1

D-T fusion energy: your first Fortran program

Fortran | Sat 17 Oct 2026 | FT-1.1 | ~25 min

In a D-T reaction, a deuteron and a triton fuse into a helium-4 nucleus and a neutron:

$$\mathrm{D} + \mathrm{T} \;\rightarrow\; {}^{4}\mathrm{He} + n + Q$$

The released energy Q comes from the mass that goes missing, $Q = \Delta m\, c^2$. You'll write a complete Fortran program that computes Q and the energy in one kilogram of D-T fuel, and see why every Fortran program starts with `implicit none`.

New to Fortran? Read the theory first.

Tasks

The starter `solution.f90` already declares all physical constants (masses in u, unit conversions). Don't change them.

1. **Compile and run the skeleton** as it is:

   ```
   gfortran -Wall -fcheck=all solution.f90 -o solution
   ./solution
   ```

   It prints one line. From now on: edit, recompile, rerun.

2. **Mass defect and Q.** Declare `real(dp)` variables `dm` and `Q`. Compute

   $$\Delta m = (m_{\mathrm{D}} + m_{\mathrm{T}}) - (m_{\mathrm{He}} + m_n), \qquad Q = \Delta m \cdot 931.494\ \mathrm{MeV/u}$$

   and print both with labels (`print *, 'Q (MeV) =', Q`).

3. **Energy per kilogram.** Declare `n_per_kg` and `E_kg`. One reaction consumes one D and one T, so the number of reactions in 1 kg of fuel is $1/\big((m_{\mathrm{D}} + m_{\mathrm{T}}) \cdot u_{\mathrm{kg}}\big)$. Compute it and $E_{\mathrm{kg}} = n_{\text{per kg}} \cdot Q \cdot (\text{J per MeV})$ in J/kg, and print both.

4. **Why `implicit none`?** Copy `solution.f90` to `typo.f90` and misspell `E_kg` in its `print` line only (e.g. `E_gk`). (a) Compile it: what does the compiler say? (b) Delete the `implicit none` line, compile and run: what happens? Answer in the two comment lines at the bottom of `solution.f90`; don't commit `typo.f90`.

Check yourself

$Q \approx 17.59$ MeV and $E_{\mathrm{kg}} \approx 3.37 \times 10^{14}$ J/kg, roughly ten million times the energy in a kilogram of coal.

Submit

Commit and push `solution.f90` (not the compiled binary).

THEORY

Fortran program structure, `implicit none`, compiling and running

The idea

Python interprets your source line by line. Fortran is **compiled**: `gfortran` first translates the whole file into machine code, checking types on the way, and then you run the result. The extra step buys speed and errors caught *before* a 10-hour run starts, not in hour 9.

Anatomy of a program

```
program circle                        ! name of the program
  use, intrinsic :: iso_fortran_env, only: dp => real64
  implicit none                       ! every variable must be declared
  real(dp), parameter :: pi = 3.141592653589793_dp   ! a named constant
  real(dp) :: r, area                                ! variables
  r = 2.0_dp
  area = pi * r**2
  print *, 'area =', area
end program circle
```

The order is fixed: `program`, then `use` statements (modules; here the 64-bit real kind), then `implicit none`, then **all declarations**, then executable statements (a declaration after the first one is an error), then `end program`. `!` starts a comment, `**` is the power, and `print *, a, b` prints with a default format (nicer formatting is FT-1.4). Fortran is **case-insensitive**: `Q` and `q` are the same variable.

Types for now

`real(dp)` is a 64-bit real (a Python `float`), and `integer` is an integer. Write real literals with the `_dp` suffix: `2.0_dp`, `1.6e-13_dp`. A literal without it is *single* precision (about 7 digits), even when you assign it to a `real(dp)` variable. Kinds are FT-1.2; for today, just put `_dp` on every real number (forgetting it on `1.66053906660e-27` silently costs about 9 digits). A `parameter` is a constant fixed at compile time: assigning to it later is a compile error.

`implicit none`

Old Fortran had **implicit typing**: an undeclared name was created automatically, as an `integer` if its first letter was `i`–`n` and as a `real` otherwise. Any typo silently becomes a brand-new variable. `implicit none` switches this off, so every name must be declared; task 4 shows what that buys. Put it in every program unit you write.

Compiling and running

In the build command of task 1, `-Wall` turns on most warnings, `-fcheck=all` adds runtime checks (slower, for development only), and `-o solution` names the executable (default `a.out`); `./solution` runs it. Compiler messages point to a line and column: `solution.f90:12:16:` means line 12, column 16. Fix the **first** error first; later ones are often knock-on effects. Recompile after every edit: running an old binary is the classic "my fix did nothing" moment.